



BSc (Hons) in Computer Science
Parallel Computing | Year 3, Semester 1, 2026

SE4060-MPI-Part1

Name :- Jananjana J.W.C – IT24104006

Repo URL:

<https://github.com/IT24104006/SE4060-MPI-Part1.git>

Table of Contents

Exercise 01: HelloMPI at 2 and 4 processes, plus messages1 and message2	4
Exercise 02: The sum and Pi runs, and the two graphs from exercise4	6
Exercise 03	8
Exercise 04	9
Exercise 05:	9
<i>Part 1: source and destination mismatch</i>	9
<i>Part 2: message2 with MPI_Bsend</i>	10
Exercise 06: Pi with MPI_ANY_SOURCE / arrival order	11
Exercise 07: Pi values and timings	12

First Set Up AWS, for instance:

The screenshot displays the AWS Management Console interface, specifically the EC2 Instances page. A green banner at the top indicates the "Successful launch of instance i-0a59da3e6d5d175f6". Below this, the "Next Steps" section provides guidance on managing the instance, including creating billing alerts, connecting to the instance, connecting to an RDS database, and creating an EBS snapshot policy.

The main content area shows the "Instance summary for i-0a59da3e6d5d175f6". Key details include:

- Instance ID:** i-0a59da3e6d5d175f6
- Public IPv4 address:** 54.175.130.136
- Instance state:** Running
- Private IPv4 address:** 172.31.19.44
- Public DNS:** ec2-54-175-130-136.compute-1.amazonaws.com
- Host name:** ip-172-31-19-44.ec2.internal
- Private IP DNS name:** ip-172-31-19-44.ec2.internal
- Instance type:** t3.small
- VPC ID:** vpc-0fbdd8d7a2e4f8b97
- Subnet ID:** subnet-0fb26878db372f35d
- Auto-assigned IP address:** 54.175.130.136
- Auto Scaling Group name:** -

Below the instance summary, the "Instances (3)" table lists the launched instances:

Name	Instance ID	Instance state	Instance type	Status checks	Availability Zone	Public IPv4 DNS
lab02	i-051577f9a1259a48	Running	t3.micro	3/3 checks passed	us-east-1a	ec2-52-207-241-40.co...
lab01-highcon...	i-08b3453366398a0d	Running	t3.small	3/3 checks passed	us-east-1a	ec2-18-234-86-216.co...
Lab08	i-0a59da3e6d5d175f6	Running	t3.small	3/3 checks passed	us-east-1a	ec2-54-175-130.136.co...

The bottom section, "All instances", provides a summary of instance alarm states, service health, and health events affecting EC2 instances.

```
[root@ec2 ~]# ssh -o StrictHostKeyChecking=no ec2-user@i-0a59da5e5d175f6175f6
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

josh@ip-172-31-19-44 ~$ cat /etc/passwd | grep josh
josh:x:1000:1000::/home/josh:/bin/bash

josh login: Wed Sep 30 22:14:39 2026 from 18.206.107.28
ec2-user@ip-172-31-19-44 ~$ echo "export PATH=/usr/lib64/openssl/bin:$PATH" >> ~/.bashrc
ec2-user@ip-172-31-19-44 ~$ echo "export LD_LIBRARY_PATH=/usr/lib64/openssl/lib64:$LD_LIBRARY_PATH" >> ~/.bashrc
ec2-user@ip-172-31-19-44 ~$ git clone https://github.com/I724104006/SE4K60-MPI-part1.git
Cloning into 'se4k60-mpi-part1'...
remote: Enumerating objects: 59, done.
remote: Counting objects: 59 (59%), done.
remote: Compressing objects: 100% (53/53), done.
remote: Total 59 (delta: 19, reused 4 (delta: 0), pack-reused 0 (from 0))
receiving objects: 100% (59/59), 15.08 KiB | 7.54 MiB/s, done.
resolving deltas: 100% (19/19), done.
ec2-user@ip-172-31-19-44 ~$ cd SE4K60-MPI-part1
ec2-user@ip-172-31-19-44 SE4K60-MPI-part1$ ls
Makefile  README.md  se4k60_mpi_message.ccc  message1.cc
ec2-user@ip-172-31-19-44 SE4K60-MPI-part1$
```

```
ec2-user@ip-172-31-19-44 SE4060-MP1-Part1]$ cat > HelloMPI.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>
int main(void)
{
    int rank;
    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    char name[MPI_MAX_PROCESSOR_NAME];
    int len;
    MPI_Get_processor_name(name, &len);
    if (rank == 0)
        printf("Hello World! From rank %d machine %s\n", rank, name);
    else
        printf("Just a normal process From rank %d machine %s\n", rank, name);
    MPI_Finalize();
    return 0;
}
EOF
```

i-0a59da3e6d5d175f6 (Lab08)

PublicIPs: 3.87.108.195 PrivateIPs: 172.31.19.44

Cloudshell Agent Toolkit for AWS Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Replace HelloMPI.c with updated one

```
aws [Alt+S] [Search] [United States (N. Virginia)] Account ID: 2506-6585-7662 vocaba/user5254362-IT24104006@my.sill.it

[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ export PATH=/usr/lib64/openmpi/bin:$PATH
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ export LD_LIBRARY_PATH=/usr/lib64/openmpi/lib:$LD_LIBRARY_PATH
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ which mpicc mpirun
/usr/lib64/openmpi/bin/mpicc
/usr/lib64/openmpi/bin/mpirun
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ echo 'export PATH=/usr/lib64/openmpi/bin:$PATH' >> ~/.bashrc
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ echo 'export LD_LIBRARY_PATH=/usr/lib64/openmpi/lib:$LD_LIBRARY_PATH' >> ~/.bashrc
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ grep openmpi ~/.bashrc
export PATH=/usr/lib64/openmpi/bin:$PATH
export LD_LIBRARY_PATH=/usr/lib64/openmpi/lib:$LD_LIBRARY_PATH
export PATH=/usr/lib64/openmpi/bin:$PATH
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ mpicc -o job1 HelloMPI.c
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ mpirun -n 2 ./job1

There are not enough slots available in the system to satisfy the 2
slots that were requested by the application:

./job1

Either request fewer slots for your application, or make more slots
available for use.

A "slot" is the Open MPI term for an allocatable unit where we can
launch a process. The number of slots available are defined by the
environment in which Open MPI processes are run:

1. Hostfile, via "slots=N" clauses (N defaults to number of
   processor cores if not provided)
2. The --host command line parameter, via a "N" suffix on the
   hostname (N defaults to 1 if not provided)
3. Resource manager (e.g., SLURM, PBS/Torque, LSF, etc.)
4. If none of the above, the --host command line parameter, or an
   RM is present, Open MPI defaults to the number of processor cores

In all the above cases, if you want Open MPI to default to the number
of hardware threads instead of the number of processor cores, use the
--use-hwthread-cpus option.

Alternatively, you can use the --oversubscribe option to ignore the
number of available slots when deciding the number of processes to
launch.

[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ mpirun --oversubscribe -n 4 ./job1
```

i-0a59da3e6d5d175f6 (Lab08)
PublicIPs: 3.87.108.195 PrivateIPs: 172.31.19.44

CloudShell Agent Toolkit for AWS Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

```
aws [Alt+S] [Search] [United States (N. Virginia)] Account ID: 2506-6585-7662 vocaba/user5254362-IT24104006@my.sill.it

There are not enough slots available in the system to satisfy the 2
slots that were requested by the application:

./job1

Either request fewer slots for your application, or make more slots
available for use.

A "slot" is the Open MPI term for an allocatable unit where we can
launch a process. The number of slots available are defined by the
environment in which Open MPI processes are run:

1. Hostfile, via "slots=N" clauses (N defaults to number of
   processor cores if not provided)
2. The --host command line parameter, via a "N" suffix on the
   hostname (N defaults to 1 if not provided)
3. Resource manager (e.g., SLURM, PBS/Torque, LSF, etc.)
4. If none of the above, the --host command line parameter, or an
   RM is present, Open MPI defaults to the number of processor cores

In all the above cases, if you want Open MPI to default to the number
of hardware threads instead of the number of processor cores, use the
--use-hwthread-cpus option.

Alternatively, you can use the --oversubscribe option to ignore the
number of available slots when deciding the number of processes to
launch.

[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ mpirun --oversubscribe -n 4 ./job1
```

i-0a59da3e6d5d175f6 (Lab08)
PublicIPs: 3.87.108.195 PrivateIPs: 172.31.19.44

CloudShell Agent Toolkit for AWS Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

```
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ export OMPI_MCA_rmaps_base_oversubscribe=1
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ mpicc -o job1 HelloMPI.c
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ mpirun -n 2 ./job1
Hello World! From rank 0 machine ip-172-31-19-44.ec2.internal
Just a normal process From rank 1 machine ip-172-31-19-44.ec2.internal
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ mpirun -n 4 ./job1
Hello World! From rank 0 machine ip-172-31-19-44.ec2.internal
Just a normal process From rank 2 machine ip-172-31-19-44.ec2.internal
Just a normal process From rank 1 machine ip-172-31-19-44.ec2.internal
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$
```

i-0a59da3e6d5d175f6 (Lab08)
PublicIPs: 3.87.108.195 PrivateIPs: 172.31.19.44

CloudShell Agent Toolkit for AWS Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

```
aws [Alt+S] [Search] [United States (N. Virginia)] Account ID: 2506-6585-7662 vocaba/user5254362-IT24104006@my.sill.it

[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ sed -i 's/char name\[30\]/char name\[MPI_MAX_PROCESSOR_NAME\]/' message1.c message2.c
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ mpicc -o messages1 message1.c
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ mpirun -n 4 ./messages1
Just a normal process From rank 0 machine ip-172-31-19-44.ec2.internal
Just a normal process From rank 2 machine ip-172-31-19-44.ec2.internal
Sending message to computer 3 from computer 1
In computer 3 the value of y is 30
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$
```

i-0a59da3e6d5d175f6 (Lab08)
PublicIPs: 3.87.108.195 PrivateIPs: 172.31.19.44

CloudShell Agent Toolkit for AWS Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

```
aws
Search [Alt+S]
[United States (N. Virginia)] Account ID: 2506-6585-7662 voclabs/user5254362=IT24104006@my.slin.it

[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ sed -i 's/print(" %d",y[r])/a\ printf("\n");' message2.cc
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ mpiexec -o message2 message2.cc
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ mpirun -n 4 ./message2
Just a normal process From rank 0 machine ip-172-31-19-44.ec2.internal
Just a normal process From rank 2 machine ip-172-31-19-44.ec2.internal
Sending message to computer 3 from computer 1
In computer 3 the value of y is printed
0 10 20 30 40 50 60 70 80 90

[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$
```

i-0a59da3e6d5d175f6 (Lab08)
PublicIPs: 3.87.108.195 PrivateIPs: 172.31.19.44

CloudShell Agent Toolkit for AWS Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

```
aws
Search [Alt+S]
[United States (N. Virginia)] Account ID: 2506-6585-7662 voclabs/user5254362=IT24104006@my.slin.it

On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    message2
    message1

nothing added to commit but untracked files present (use "git add" to track)
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ git push origin main
Username for 'https://github.com': IT24104006
Password for 'https://IT24104006@github.com':
remote: Invalid username or token. Password authentication is not supported for Git operations.
fatal: Authentication failed for 'https://github.com:IT24104006/SE4060-MPI-Part1.git/'
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ git push origin main
Username for 'https://github.com': IT24104006
Password for 'https://IT24104006@github.com':
Enumerating objects: 0, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 2 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 623 bytes | 623.00 KiB/s, done.
Total 5 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com:IT24104006/SE4060-MPI-Part1.git
   eb669f2..219aac2 main -> main
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$
```

i-0a59da3e6d5d175f6 (Lab08)
PublicIPs: 3.87.108.195 PrivateIPs: 172.31.19.44

CloudShell Agent Toolkit for AWS Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Exercise 02: The sum and Pi runs, and the two graphs from exercise4

```
aws
Search [Alt+S]
[United States (N. Virginia)] Account ID: 2506-6585-7662 voclabs/user5254362=IT24104006@my.slin.it

#include <stdio.h>
#include <mpi.h>

#define N 100000000LL

int main(void)
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();

    /* each rank sums its own chunk of 1..N */
    long long start = (N * rank) / size + 1;
    long long end = (N * (rank + 1)) / size;
    long long local = 0;
    for (long long i = start; i <= end; i++)
        local += i;

    if (rank == 0) {
        long long total = local;
        for (int src = 1; src < size; src++) {
            MPI_Recv(&total, 1, MPI_LONG_LONG, src, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            total += total;
        }
    }

    printf("Sum: %lld\n", total);
    MPI_Finalize();
    return 0;
}
```

i-0a59da3e6d5d175f6 (Lab08)
PublicIPs: 3.87.108.195 PrivateIPs: 172.31.19.44

CloudShell Agent Toolkit for AWS Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

```
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ export MPI_MCA_rmaps_base_oversubscribe=1
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ mpicc -o2 -o sum_mpi sum_mpi.c
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ mpirun -n 1 ./sum_mpi
Processes: 1 Sum: 5000000500000000 Time: 0.003270 s
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ mpirun -n 2 ./sum_mpi
Processes: 2 Sum: 5000000500000000 Time: 0.003271 s
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ mpirun -n 4 ./sum_mpi
Processes: 4 Sum: 5000000500000000 Time: 0.003406 s
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ mpirun -n 8 ./sum_mpi
Processes: 8 Sum: 5000000500000000 Time: 0.000933 s
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$
```

i-0a59da3e6d5d175f6 (Lab08)
PublicIPs: 3.87.108.195 PrivateIPs: 172.31.19.44

CloudShell Agent Toolkit for AWS Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

```
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ cat > run_sum.sh << 'EOF'
#!/bin/bash
export OMPI_MCA_rmaps_base_oversubscribe=1
echo "procs,run,time" > sum_results.csv
for p in 1 2 4 8; do
  for i in 1 2 3 4 5 6 7 8 9 10; do
    t=$(mpirun --bind-to none -n $p ./sum_mpi | awk '{print $(NF-1)}')
    echo "$p,$i,$t" >> sum_results.csv
  done
done
done
EOF
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ chmod +x run_sum.sh
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ ./run_sum.sh
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ awk -F ',' 'NR>1 {s[$1]+=$3; n[$1]++; if(!($1 in m){s<m[$1] m[$1]=$3} END {for(p in s) printf "%s procs: avg %.6f min %.6f\n", p, s[p]/n[p], m[p]}' sum_results.csv | sort -n
1 procs: avg 0.003257 min 0.003226
2 procs: avg 0.003293 min 0.003268
4 procs: avg 0.003683 min 0.003333
8 procs: avg 0.002634 min 0.000679
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$
```

i-0a59da3e6d5d175f6 (Lab08)
PublicIPs: 3.87.108.195 PrivateIPs: 172.31.19.44

CloudShell Agent Toolkit for AWS Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

```
int main(void)
{
    int rank, size;
    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();

    /* each rank sums its own chunk of 1..N */
    long long start = (N * rank) / size + 1;
    long long end = (N * (rank + 1)) / size;
    long long local = 0;
    for (long long i = start; i <= end; i++)
        local += i;

    long long total = 0;
    if (rank == 0) {
        long long part;
        total = local;
        for (int src = 1; src < size; src++) {
            MPI_Recv(&part, 1, MPI_LONG_LONG, src, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            total += part;
        }
    }
    printf("NR=1 {s[$1]+=$3; n[$1]++; if(!($1 in m){s<m[$1] m[$1]=$3} END {for(p in s) printf \"%s procs: avg %.6f min %.6f\n\", p, s[p]/n[p], m[p]}' sum_results.csv | sort -n\n");
    1 procs: avg 0.006510 min 0.006479
    2 procs: avg 0.006508 min 0.006469
    4 procs: avg 0.007408 min 0.006515
    8 procs: avg 0.005758 min 0.001856
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$
```

i-0a59da3e6d5d175f6 (Lab08)
PublicIPs: 3.87.108.195 PrivateIPs: 172.31.19.44

CloudShell Agent Toolkit for AWS Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

```
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ nproc
2
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ lscpu | grep -E "Model name|^CPU\(\s\)\[Thread|Core|MHz"
CPU(s): 2
Model name: Intel(R) Xeon(R) Platinum 8259CL CPU @ 2.50GHz
Thread(s) per core: 2
Core(s) per socket: 1
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ echo "--- one alone"
--- one alone
mpirun -n 1 ./sum_mpi
echo "--- two at once"
for i in 1 2; do mpirun -n 1 ./sum_mpi & done; wait
--- one alone
Processors: 1 Sum: 50000005000000 Time: 0.006475 s
--- two at once
[1] 34770
[2] 34771
Processors: 1 Sum: 50000005000000 Time: 0.009991 s
Processors: 1 Sum: 50000005000000 Time: 0.006481 s
[1]- Done mpirun -n 1 ./sum_mpi
[2]+ Done mpirun -n 1 ./sum_mpi
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$
```

i-0a59da3e6d5d175f6 (Lab08)
PublicIPs: 3.87.108.195 PrivateIPs: 172.31.19.44

CloudShell Agent Toolkit for AWS Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Exercise 2: Sum of 1 to 10 000 000

The numbers 1 to 10,000,000 are distributed in chunks of equal size to each process. Each process sums the numbers in its chunk and then sends the result to the process with rank 0 with `MPI_Send`. This master process then accumulates the results from all other processes by receiving them through `MPI_Recv`. For each run only the correct result 50000005000000 was printed. My first implementation took only about 3 ms and thus could not be measured reliably, thus the final version of the code was changed to make 50 measurements of the summation procedure and report the total time spent. This brings the duration of a single run to about 0.2 s and each result is an average over 10 runs.

```
Thread(s) per core: 2
Size(s) per socket: 1
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ echo "---- one alone"
mpirun -n 1 ./sum_mpi
echo P== two at once"
for i in 1 2; do mpirun -n 1 ./sum_mpi & done; wait
--- one alone
Processes: 1 Sum: 50000005000000 Time: 0.006475 s
--- two at once
[1] 34770
[2] 34771
Processes: 1 Sum: 50000005000000 Time: 0.009991 s
Processes: 1 Sum: 50000005000000 Time: 0.006481 s
[1]- Done mpirun -n 1 ./sum_mpi
[2]+ Done mpirun -n 1 ./sum_mpi
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ vmstat 1 10
procs-----memory-----swap-----io-----system-----cpu-----
r b swpd free buff cache si so bi bo in cs us sy id wa st
1 0 0 947996 2168 804132 0 0 0 32 76 71 154 1 1 98 0 0
0 0 0 947996 2168 804132 0 0 0 0 70 64 0 0 100 0 0
0 0 0 947996 2168 804176 0 0 0 0 45 58 0 0 100 0 0
0 0 0 947996 2168 804176 0 0 0 0 55 48 0 0 100 0 0
0 0 0 947996 2168 804176 0 0 0 4 74 68 0 0 100 0 0
0 0 0 947996 2168 804176 0 0 0 0 61 55 0 0 100 0 0
0 0 0 947996 2168 804176 0 0 0 0 68 64 0 0 100 0 0
0 0 0 947996 2168 804176 0 0 0 0 57 50 0 0 100 0 0
0 0 0 947996 2168 804176 0 0 0 0 63 62 0 0 100 0 0
0 0 0 947996 2168 804176 0 0 0 0 58 52 0 0 100 0 0
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$
```

i-0a59da3e6d5d175f6 (Lab08)
PublicIPs: 3.87.108.195 PrivateIPs: 172.31.19.44

CloudShell Agent Toolkit for AWS Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

```
Processes: 1 Sum: 50000005000000 Time: 0.204229 s
Processes: 2 Sum: 50000005000000 Time: 0.202609 s
Processes: 4 Sum: 50000005000000 Time: 0.251532 s
Processes: 8 Sum: 50000005000000 Time: 0.200718 s
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ for i in 1 2; do mpirun -n 1 ./sum_mpi & done; wait
[1] 35164
[2] 35165
Processes: 1 Sum: 50000005000000 Time: 0.405762 s
Processes: 1 Sum: 50000005000000 Time: 0.402408 s
[1]- Done mpirun -n 1 ./sum_mpi
[2]+ Done mpirun -n 1 ./sum_mpi
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$
```

i-0a59da3e6d5d175f6 (Lab08)
PublicIPs: 3.87.108.195 PrivateIPs: 172.31.19.44

CloudShell Agent Toolkit for AWS Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Exercise 03

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define N 10000000LL

int main(void)
{
    int rank, size;
    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();

    /* each rank throws its share of the N points */
    long long start = (N * rank) / size;
    long long end = (N * (rank + 1)) / size;
    unsigned int seed = 12345 + (unsigned int)rank; /* different stream per rank */
    long long local = 0;
    for (long long i = start; i < end; i++) {
        double x = (double)rand_r(&seed) / RAND_MAX;
        double y = (double)rand_r(&seed) / RAND_MAX;
        if (x * x + y * y <= 1.0)
            local++;
    }

    MPI_Finalize();
    return 0;
}
```

i-0a59da3e6d5d175f6 (Lab08)
PublicIPs: 3.87.108.195 PrivateIPs: 172.31.19.44

CloudShell Agent Toolkit for AWS Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Exercise 3: Monte Carlo Pi

Every process randomly selects one million out of the available 10,000,000 points, and tallies the number of hits that have been within the unit square. Each process relays its tally to the zeroth process which uses it to estimate Pi as $4 \times \text{hits} / 10,000,000$. Every process is using a different random seed, since otherwise the runs would not have been independent. The resulting estimates were 3.141386, 3.141265, 3.140876 and 3.141124 when using 1, 2, 4 and 8 processes, all of which are within about 0.0005 of the true value. They are not equal to each other since each process is using an independent random stream.

Exercise 04

```
Collecting kiwisolver==1.3.1
  Downloading kiwisolver-1.4.7-cp39-cp39-manylinux_2_12_x86_64.manylinux2010_x86_64.whl (1.6 MB)
Collecting cycler==0.10
  Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Collecting numpy==1.23
  Downloading numpy-2.0.2-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (19.5 MB)
Collecting importlib-resources==3.2.0
  Downloading importlib_resources-6.5.2-py3-none-any.whl (37 kB)
Collecting fonttools==4.22.0
  Downloading fonttools-4.60.2-cp39-cp39-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (4.8 MB)
Collecting packaging==20.0
  Downloading packaging-26.3-py3-none-any.whl (129 kB)
Collecting contourpy==1.0.1
  Downloading contourpy-1.3.0-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (321 kB)
Collecting pyparsing==2.3.1
  Downloading pyparsing-3.3.3-py3-none-any.whl (126 kB)
Collecting zipp==3.1.0
  Downloading zipp-3.23.1-py3-none-any.whl (10 kB)
Requirement already satisfied: six>=1.5 in /usr/lib/python3.9/site-packages (from python-dateutil>=2.7->matplotlib) (1.15.0)
Installing collected packages: zipp, numpy, pyparsing, pillow, kiwisolver, importlib-resources, fonttools, cycler, contourpy, matplotlib
Successfully installed contourpy-1.3.0 cycler-0.12.1 fonttools-4.60.2 importlib-resources-6.5.2 kiwisolver-1.4.7 matplotlib-3.9.4 numpy-2.0.2 packaging-26.3 pillow-11.3.0 pyparsing-3.3.3 zipp-3.23.1
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$
```

```
run_sum.sh sum_mpi sum_mpi.c sum_results.csv
Exercise 3:
$ mpiexec -n 1 mpi.c pi_results.csv run_pi.sh
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ cp exercise4/exercise4/plot_results.py /tmp/plot_results.py
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ rm -rf exercise4
mkdir exercise4
mv /tmp/plot_results.py exercise4/
cd exercise4
[ec2-user@ip-172-31-19-44 exercise4]$ ls
plot_results.py
[ec2-user@ip-172-31-19-44 exercise4]$ python3 plot_results.py
Exercise 2: Sum of 1..10,000,000
process avg time(s) stdev(s) speedup
1 0.208253 0.019474 1.00
2 0.204026 0.002497 1.02
4 0.222968 0.021143 0.93
8 0.195537 0.017182 1.07
Exercise 3: Monte Carlo Pi (10,000,000 samples)
process avg time(s) stdev(s) speedup
1 0.142100 0.000068 1.00
2 0.090051 0.000320 1.58
4 0.124789 0.021783 1.14
8 0.094835 0.022428 1.50
Saved time vs processes.png and speedup.png
[ec2-user@ip-172-31-19-44 exercise4]$ ls
plot_results.py speedup.png time_vs_processes.png
[ec2-user@ip-172-31-19-44 exercise4]$
```

Exercise 4: Time and speedup

While the Pi program speeded up nicely from 1 to 2 processes (0.142 s to 0.090 s), more processes gave no benefit – and even with 8 processes the result was the same as with 2. The sum showed barely any speedup either, varying between roughly 0.93 and 1.07x for all process counts. This appears to be due to the hardware: lscpu reports 2 vCPUs for the t3.small, but these turn out to be 2 hardware threads on a single core. To confirm, I launched 2 independent sum calculations at the same time – and each took about 0.40 s, vs 0.20 s for a single process. The increased runtime is similar to what we’d see on a single core with 2 threads: the second thread in the Pi calculation has more idle cycles (due to random-number calls and a conditional branch) and can’t utilise the core as fully as the first, but the tight loop in the sum calculation keeps it busy. Beyond 2, processes contend for the single core.

Exercise 05:

Part 1: source and destination mismatch

```
aws [Search] [Alt+S] Ask Amazon Q United States (N. Virginia) Account ID: 2506-6585-7662 voclabs/user5254362=it724104

[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ cat > mismatch.c << 'EOF'
#include <stdio.h>
#include <mpi.h>

int main(void)
{
    int rank, x = 30, y = 0;
    MPI_Status status;
    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 1) {
        printf("Rank 1: sending to rank 3\n");
        fflush(stdout);
        MPI_Ssend(&x, 1, MPI_INT, 3, 0, MPI_COMM_WORLD);
        printf("Rank 1: send completed\n");
    } else if (rank == 3) {
        printf("Rank 3: waiting for a message from rank 2\n");
        fflush(stdout);
        MPI_Recv(&y, 1, MPI_INT, 2, 0, MPI_COMM_WORLD, &status);
        printf("Rank 3: received %d\n", y);
    }

    MPI_Finalize();
    return 0;
}
EOF
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ mpicc -o mismatch mismatch.c
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$ timeout 10 mpirun -n 4 ./mismatch; echo "exit code: $?"
Rank 1: sending to rank 3
Rank 3: waiting for a message from rank 2
exit code: 124
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$

i-0a59da3e6d5d175f6 (Lab08)
Public IPs: 54.91.216.71 Private IPs: 172.31.19.44

CloudShell Agent Toolkit for AWS Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

[ec2-user@ip-172-31-19-44 exercise5]$ timeout 10 mpirun -n 4 ./mismatch; echo "exit code: $?"
Rank 3: waiting for a message from rank 2
Rank 1: sending to rank 3
exit code: 124
[ec2-user@ip-172-31-19-44 exercise5]$
```

In this program, rank 1 sends to rank 3, but rank 3 is waiting for a message from rank 2 which is never sent. The program deadlocks. Rank 3 is blocked in `MPI_Recv` waiting for a matching sender, and rank 1 is blocked in `MPI_Ssend` since a synchronous send will only complete when a matching receive has started. MPI does not detect this as an error since it cannot know that the program is incorrect. I have used timeout 10 to terminate it, and the exit code 124 in the screenshot indicates that the timeout has occurred.

Part 2: message2 with `MPI_Bsend`

```

aws [Alt+S] Ask Amazon Q United States (N. Virginia) Account ID: 2506-6585-7662
voclabs/user5254362=IT24104006@my.sllt.it

#include <stdlib.h>
#include <mpi.h>

int main(void)
{
    int rank;
    MPI_Status status;
    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    char name[MPI_MAX_PROCESSOR_NAME];
    int len;
    MPI_Get_processor_name(name, &len);
    int x[10], y[10];

    if (rank == 1) {
        for (int r = 0; r < 10; r++)
            x[r] = 10 * r;

        int bufsize = 10 * sizeof(int) + MPI_BSEND_OVERHEAD;
        void *buf = malloc(bufsize);
        MPI_Buffer_attach(buf, bufsize);

        printf("Sending message to rank 3 from rank 1 using MPI_Bsend\n");
        MPI_Bsend(x, 10, MPI_INT, 3, 0, MPI_COMM_WORLD);

        /* overwrite the send array straight away */
        for (int r = 0; r < 10; r++)
            x[r] = -1;
    }

    return 0;
}
EOF
[ec2-user@ip-172-31-19-44 SE4060-MPI-Part1]$

```

MPI_Bsend copies the data into a buffer allocated by MPI_Buffer_attach function and immediately returns to the caller. To verify that, rank 1 updated its send array with -1 immediately after sending. However, rank 3 still got the original values from 0, 10, 20 until 90, since the data was already copied to a buffer.

Exercise 06: Pi with MPI_ANY_SOURCE / arrival order

```

aws [Alt+S] Ask Amazon Q United States (N. Virginia) Account ID: 2506-6585-7662
voclabs/user5254362=IT24104006@my.sllt.it

[ec2-user@ip-172-31-19-44 exercise6]$ mpicc -O2 -o pi_any_source pi_any_source.c
[ec2-user@ip-172-31-19-44 exercise6]$ for p in 1 2 4 8; do mpirun --bind-to none -n $p ./pi_any_source; done
Arrival order:
Processes: 1 Pi: 3.141386 Time: 0.145219 s
Arrival order: 1
Processes: 2 Pi: 3.141265 Time: 0.098480 s
Arrival order: 1 2 3
Processes: 4 Pi: 3.140876 Time: 0.098590 s
Arrival order: 1 2 3 5 6 7 4
Processes: 8 Pi: 3.141124 Time: 0.083632 s

```

Timing comparison

```

[ec2-user@ip-172-31-19-44 exercise6]$ cat > run_pi_any.sh << 'EOF'
#!/bin/bash
export OMPI_MCA_rmaps_base_oversubscribe=1
echo "procs,run,time" > pi_any_results.csv
for p in 1 2 4 8; do
    for r in 1 2 3 4 5 6 7 8 9 10; do
        t=$(mpirun --bind-to none -n $p ./pi_any_source | tail -n 1 | awk '{print $(NF-1)}')
        echo "$p,$r,$t" >> pi_any_results.csv
    done
done
EOF
chmod +x run_pi_any.sh
[ec2-user@ip-172-31-19-44 exercise6]$ wc -l pi_any_results.csv
29 pi_any_results.csv
4,6,0.172166
4,7,0.170440
4,8,0.148594
[ec2-user@ip-172-31-19-44 exercise6]$ ./run_pi_any.sh
awk -F, 'NR>1 {s1[s1]++}; END {for(p in s) printf "%s procs: avg %.6f (n=%d)\n", p, s[p]/n[p], n[p]]}' pi_any_results.csv | sort -n
1 procs: avg 0.147020 (n=10)
2 procs: avg 0.099734 (n=10)
4 procs: avg 0.128038 (n=10)
8 procs: avg 0.101849 (n=10)
[ec2-user@ip-172-31-19-44 exercise6]$

```

Instead of getting the result from the rank 1, then rank 2, and so on, rank 0 now gets the result from whoever finishes first (MPI_ANY_SOURCE). The program prints the result in the order it was received. If we run the 8-process version several times, we will get a different order each time. The Pi values are the same as in Exercise 3 (3.141386, 3.141265, 3.140876 and 3.141124), since the hit count totals are independent of the order in which messages are received. The average times are 0.147, 0.100, 0.128 and 0.102 s for 1, 2, 4 and 8 processes. Previously they were 0.142, 0.090, 0.125 and 0.095 s. The difference is small and is also seen for 1 process (where no receive happens) so I would attribute it to run-to-run variation rather than a significant effect due to MPI_ANY_SOURCE.

Exercise 07: Pi values and timings

```
[ec2-user@ip-172-31-19-44 exercise7]$ mpicc -O2 -o pi_bsend pi_bsend.c
[ec2-user@ip-172-31-19-44 exercise7]$ for p in 1 2 4 8; do mpirun --bind-to none -n $p ./pi_bsend; done
Arrival order:
Processes: 1 Pi: 3.141386 Time: 0.144573 s
Arrival order: 1
Processes: 2 Pi: 3.141265 Time: 0.102210 s
Arrival order: 3 1 2
Processes: 4 Pi: 3.140876 Time: 0.144270 s
Arrival order: 6 1 2 3 4 5 7
Processes: 8 Pi: 3.141124 Time: 0.073308 s
```

Timing comparison

```
[ec2-user@ip-172-31-19-44 exercise7]$ cat > run_pi_bsend.sh << 'EOF'
#!/bin/bash
export OMPI_MCA_rmaps_base_oversubscribe=1
echo "procs,run,time" > pi_bsend_results.csv
for p in 1 2 4 8; do
  for r in 1 2 3 4 5 6 7 8 9 10; do
    t=$(mpirun --bind-to none -n $p ./pi_bsend | tail -n 1 | awk '{print $(NF-1)}')
    echo "$p,$r,$t" >> pi_bsend_results.csv
  done
done
EOF
[ec2-user@ip-172-31-19-44 exercise7]$ chmod +x run_pi_bsend.sh
[ec2-user@ip-172-31-19-44 exercise7]$ ./run_pi_bsend.sh
awk -F, 'NR>1 {s[$1]+=$3; n[$1]++;} END {for(p in s) printf "%s procs: avg %.6f (n=%d)\n", p, s[p]/n[p], n[p]}' pi_bsend_results.csv | sort -n
1 procs: avg 0.144340 (n=10)
2 procs: avg 0.093329 (n=10)
4 procs: avg 0.137263 (n=10)
8 procs: avg 0.106812 (n=10)
[ec2-user@ip-172-31-19-44 exercise7]$
```

This is the Exercise 6 program, in which each worker sends its count through an attached buffer using the `MPI_Bsend` call. The Pi values remain the same, while the average times are given by 0.144, 0.093, 0.137 and 0.107 s, which is approximately the same as in Exercise 6. As each message contains only one number, the buffering overhead is practically invisible. However, the buffering does offer an advantage of being able to reuse the sender's variable right away, which otherwise would have to wait for the receiver to get the message.

 **SE4060-MPI-Part1** Public template

 forked from [SLIIT-FacultyOfComputing/SE4060-MPI-Part1](#)

 Pin  Watch 0  Fork

 main  1 Branch  0 Tags

  Add file  Code

This branch is **8 commits ahead of** [SLIIT-FacultyOfComputing/SE4060-MPI-Part1:main](#)  Contribute  Sync fork

 **IT24104006** Exercise 7: Monte Carlo Pi with MPI_ANY_SOURCE and MPI_Bsend 20835ee · 13 minutes ago  27 Commits

 Archer2OpenMP	Create archer.job	5 years ago
 exercise1	Organize solutions into per-exercise folders	1 hour ago
 exercise2	Exercise 4: time and speedup graphs for sum and Pi	50 minutes ago
 exercise3	Organize solutions into per-exercise folders	1 hour ago
 exercise4	Exercise 4: time and speedup graphs for sum and Pi	50 minutes ago
 exercise5	Exercise 5: mismatched send/receive deadlock and Bsend ver...	36 minutes ago
 exercise6	Exercise 6: Monte Carlo Pi with MPI_ANY_SOURCE	21 minutes ago
 exercise7	Exercise 7: Monte Carlo Pi with MPI_ANY_SOURCE and MPI_B...	13 minutes ago
 .gitignore	Exercise 7: Monte Carlo Pi with MPI_ANY_SOURCE and MPI_B...	13 minutes ago
 Archer2.md	Update Archer2.md	5 years ago

=====END=====